Presentation guide: "What does the SDN controller actually see?"

For a 12–15 minute supervisor or panel meeting. Companion files: `paper/main.pdf` (IEEE journal format, 7 pages, authors Adhyan Jain, Atharva Sheersh Pandey, Kshitij Manikshete, VIT Vellore), `README.md` (how to reproduce) and `reproduce.sh`. Open the paper PDF next to this guide: reviewers trust a number more when you can point at its table.

The 30-second pitch

"SDN intrusion-detection papers say the controller has a global view, but they are scored on flow features computed at the victim host. InSDN's authors also recorded their ONOS controller's OpenFlow traffic, and nobody seems to have looked at it. I parsed all 44 captures. The controller never installed or reported a 5-tuple flow entry: only MAC-pair and `eth_type` entries. It does see a dense PacketIn stream, and models built only from that reach 0.946 balanced accuracy on unseen captures. But I also found that this number can't be read as attack-detection accuracy, because of how InSDN was recorded. I repeated the granularity check on a second controller application, Ryu's stock learning switch, and it also installs only MAC-pair entries. So the contribution is the measurement, a reproducible pipeline, and a validity analysis that tells the next person what InSDN can and cannot support."

The things you want the audience to leave with

1. **A real, checkable measurement:** 0 of 44 captures, 0 of 3,827 FlowMods, ever carry IP or transport match fields.
2. **It is not only ONOS:** Ryu's unmodified `simple_switch_13` (5 runs, 2,982 requests, none failed) sent 91 FlowMods: 86 MAC-pair, 5 table-miss, none with IP or transport fields. Scope: one more application, not OpenDaylight or production.
3. **A reproducible pipeline** with its own error rate reported (12 KB lost of 7.38M messages), hashes of every capture, and `reproduce.sh`, which regenerates the result files and compares them with the committed ones.
4. **You found the weakness yourself** (trivial rule 0.995; tiny session probes) and say what would fix it. This is what makes a reviewer trust the rest.

What is new since the first version of the paper

Addition	Where	Number
Second controller application (stock Ryu)	live section	91 FlowMods: 86 MAC-pair, 5 table-miss, 0 with IP/transport fields; 71, 69, 76, 76, 76 PacketIns per run
Host-side flow-CSV contrast (XGBoost, GPU, 10 seeds)	results	BAcc 0.9998 ± 0.00003 on 343,889 flows; rows of one session on both sides of the split, so it is not comparable to the controller-view numbers
Threat model and scope	new section	defender sees only the control channel; no attacks on the controller itself
Related work, systematic	section 2 + appendix	LAN-SDN-NIDS ablation (link-fabrication F1 0.97 to 0.19 without OpenFlow features); UKMDDoSDN's two PCAPs have no OpenFlow packets (0 of 309,791)
Deployment discussion	discussion	a forwarding change moves PacketIn volume 2–3 orders of magnitude, so models must be recalibrated
Reproducibility	`reproduce.sh`, `ofgap/compare_results.py`, `paper/verify_numbers.py`	features bit-identical (264 files); FlowMod, review-check, live-study and stock-Ryu results identical; paper numbers checked against JSONs

Possible sixth contribution point for the paper's list: "A reproducibility package (hashes, one-command regeneration, and an automatic check of the quoted numbers) and a second-controller check." Say it only if the professor asks what is new; it is a strength, not the headline.

Timeline (13 min)

Min	Slide	Goal
0–1	1 Question	Frame: benchmark features vs what a controller receives
1–3	2–3 Gap + PacketIn stream	Land the measurement; show the table once, don't read it
3–5	4 Forwarding case study	Show it as a *design* finding; state the confound before anyone asks
5–6	5 Method	One sentence per step; stress "no capture in both train and test"
6–8	6–7 Results + generalisation	Say "feasibility", not "detection accuracy"
8–11	8 Validity	Spend the most time here. This is your credibility slide
11–12	9 Contributions	Read the five items, one line each
12–13	10 Questions	Hand over; keep the Q&A table open

Order of honesty (say these unprompted, in this order)

1. "The 0.946 is capture-grouped, so it is optimistic. Variants of one scenario can sit on both sides of a fold."
2. "Within sessions, a trivial 'any IP PacketIn' rule scores 0.995 under my strict labels. So I do not claim detection accuracy."
3. "My first Ryu run showed only a 4× PacketIn difference because my learning switch shadowed TCP with ping/ARP entries. I fixed the app and reran: per-flow forwarding sends ~10,400 TCP PacketIns per run vs ~12 for MAC-pair, in every paired run (10 of 10; also on a 3-switch chain). One run had

41 failed requests; it did not reproduce in 59 later runs and the cause is not established."

4. "Novelty is narrow: inside InSDN, controller-observable only. LAN-SDN-NIDS (2026) already builds a control-plane dataset."

Phrases to use / avoid

Say	Don't say
"feasibility", "separates attack captures from benign captures"	"my detector achieves 94.6% accuracy"
"as far as my searches show"	"nobody has ever" / "first to"
"the controller never installed or reported a 5-tuple entry"	"SDN controllers can't see flows" (it is ONOS-in-InSDN specific)
"per-flow forwarding sent ~10,000 TCP PacketIns per run vs ~12 for MAC-pair in this testbed"	"granularity causes a 1000× PacketIn increase in SDN" (one switch, one app)
"a visibility study, not a detection study" (for the Ryu testbed)	"I validated this live"

Hard questions and what to answer

Question	Answer (short)
"So does it detect attacks?"	Not proven. It separates InSDN's attack captures from benign ones. A trivial rule beats it within sessions under strict labels. Fixing this needs benign and attack traffic interleaved in one session.
"Why not just report the 99% from the CSV papers?"	Those use features a controller never receives, so they don't describe a deployable controller-side detector. That gap is the paper.
"Logistic regression matches your MLP: what's the model contribution?"	None. The contribution is the measured view, the pipeline and the validity analysis.
"Is 10 seeds enough?"	The seeds only vary folds and initialisation on the same 44 captures; the intervals are not over new testbeds. The cluster bootstrap over captures is wider and is in the paper.
"Why is the PacketIn-rate baseline so bad?"	Plausibly because benign sessions also generate heavy PacketIn volume, but I can't verify it: the Normal group has no target-side captures.
"What would make this a stronger paper?"	A second testbed with attack and benign traffic interleaved, and detection evaluated under each forwarding mode.
"Which venue?"	As it stands: a workshop/conference or a lower-quartile journal as a measurement-and-validity paper. A stronger journal needs the interleaved testbed.
"Is this just Ryu being simple?"	Yes for the forwarding mode result: it follows from the application. That is the point: the observable depends on the application, and ONOS `fwd` and Ryu's stock app both install MAC-pair entries.
"Why does the host-side baseline score 0.9998?"	Row-level splits put flows of one session on both sides, and those features are not available to a controller. It shows the usual benchmark protocol is easy, not that the controller view is worse.

Before the meeting (checklist)

- [x] Live rerun with the corrected app is done (`results/live_modes_v4_one10.json`, `live_modes_v4_linear3.json`); the paper uses it.
- [ ] Open `paper/main.pdf` and this guide side by side; know where Tables cv, family, cross and confound are.
- [ ] Re-read slide 8 twice. It is the one you will be questioned on.
- [ ] Know these numbers cold: 44 captures, 3,827 FlowMods, 0.946, 0.616, 0.995, 6 windows from 2 captures, 10,422 vs 12.3, 153.1 → 36.9, 3-switch 10,881 vs 18.0.
- [ ] Be ready to answer "what is not done": see the limitations table below.
- [ ] Do not claim detection accuracy; the interleaved benign+attack study has not been run.

If something goes wrong

- A number is questioned: open `results/<file>.json` named under the slide; every number comes from one.
- You don't know: say "I measured X but not Y; Y is future work." Never improvise a number.

Part 2: Exactly what goes in, what comes out, where it is stored

Answer order if the professor says "walk me through the data": **raw file → parser → windows → features → model → result file → paper table**. All paths below are relative to the repo root `/home/adhyan/Desktop/Proteus`.

Stage-by-stage table

#	Stage	Input (exactly)	Code	Output (exactly)	Stored at
0	Dataset	InSDN PCAP archives from UCD (`Metsplotable-2_Group.zip`, `OVS_Group.zip`, `Normal_Group.zip`); 44 of the files are `*ONOS*` captures of the **controller↔switch OpenFlow 1.3 channel** (TCP 6653), 11 benign + 33 attack	download, `unzip -t`, SHA-256	the extracted `.pcap`/`.pcapng` files	`data/raw/insdn_pcaps/ex/` (gitignor hashes `data/raw/insdn_pcaps/ONOS_CAPTUR` provenance and URLs in `data/MANI`
1	OpenFlow parse	one pcap	`ofgap/ofparse.py` (`of_messages`): TCP reassembly per direction, dedup by sequence number, resync on corruption	a stream of `(timestamp, direction, type, body)` OpenFlow messages: PACKET_IN, PACKET_OUT, FLOW_MOD, FLOW_REMOVED, stats replies	in memory; parser error counts (mes desyncs, dropped bytes) go into `feats_v4/meta.json`
2	Labels	the InSDN CSV flow table (attack flows by Flow ID) plus the list of attack-only IPs	`ofgap/attacker_ips.py`, `ctrlfeat._label`	each PACKET_IN is tagged `att_csv` (matches a CSV attack flow), `att_ip` (attack-only IP, no CSV match), `norm_csv`, `unm_ip` (IP not in CSV), `nonip` (ARP/LLDP), `unparsed`	counts per window in `feats_v4/cNNN_W5_lab.npy`
3	Windowing + features	the message stream, window length W (1 s or 5 s)	`ofgap/ctrlfeat.py` (`extract`)	per window **22 numbers**: counts of PACKET_IN/OUT, FLOW_MOD, FLOW_REMOVED, stats replies; distinct src/dst MAC and IP, destination ports; TCP/UDP/ICMP/ARP/SYN fractions; mean frame length; entropy of src IP and dst port; flow-stats entry count and packet/byte deltas	`feats_v4/cNNN_W5.npy` (shape wind `..._lab.npy` (windows×6 label cour `..._bg.npy` (same features with atta matched PACKET_INs removed, for confound probe), `feats_v4/meta.js` capture inventory). Directory is gitign regenerate with `ctrlfeat.py <pcap_ <out_dir>`
4	History	the 22-feature rows	`exp_ctrl.with_history`	each row becomes the current + 3 previous windows (H=4, log1p scaled) = 88 inputs, causal (no future windows)	in memory
5	Models	88-feature rows and window labels `y` (1 = at least one CSV-matched attack PACKET_IN; windows with unmatched attack-like IPs are dropped as ambiguous under "strict" labels)	`ofgap/exp_ctrl2.py`: MLP, logistic regression, XGBoost (GPU), single-feature thresholds	a 0/1 prediction per window; recall, FPR, balanced accuracy per fold and seed	in memory
6	Evaluation	the predictions	`exp_ctrl2.py`	A) 5-fold CV **grouped by capture file** (a file is never in train and test); B) leave-one-attack-family-out; C) cross-deployment (Metasploitable↔OVS); D) session-confound probe; E) label-permutation control; 10 seeds, mean ± 95% CI, capture-cluster bootstrap	`results/v4_strict_W1.json`, `v4_strict_W5_xgb.json`, `v4_iprule` `v4_iprule_W5.json`

#	Stage	Input (exactly)	Code	Output (exactly)	Stored at
7	Validity checks	window counts	`ofgap/review_checks.py`	trivial rules ("any PacketIn", "any IP PacketIn") scored within attack sessions	`results/review_checks.json`
8	Granularity gap	the same 44 pcaps	`ofgap/gap_measure.py`, `flowmod_check.py`	which match fields FLOW_MODs and flow-stats replies carry (0 of 3,827 FlowMods have IP/transport match fields); PacketIns per target packet	`results/gap_attack_v2.json`, `results/flowmod_matches.json`
9	Live forwarding study	**nothing from InSDN.** Ryu app `live/ryu_fwd.py` in 3 modes + Mininet (1 OVS switch, h1–h5 clients, h6 HTTP server); benign HTTP downloads and pings only	`live/run_benign.py` (needs root), sniffer on loopback port 6653	one control-channel pcap per run, client logs	`live/campaign2/`, `live/out_linear` `live/out_stock/` (hashes of all in `capture_hashes.sha256`)
10	Live analysis	those pcaps	`live/live_table.py`	per-mode TCP PACKET_IN counts, FLOW_MOD counts, entries per 1 s stats reply, mean ± 95% CI over 10 runs, for one switch and for a 3-switch chain; stock Ryu FlowMod check (`ofgap/flowmod_stock.py`)	`results/live_modes_v4_one10.json` `live_modes_v4_linear3.json`, `stock_ryu_flowmods.json`, tables `paper/live_table*.tex`
10b	Host-side contrast	InSDN CSV (343,889 flows, 78 features)	`ofgap/hostside_baseline.py` (XGBoost, GPU)	BAcc 0.9998 ± 0.00003, 10 seeds	`results/hostside_xgb.json`
11	Paper and UI	the result JSONs	`ofgap/make_tables.py`, `live/live_table.py`; `api/server.py` `/api/controller-telemetry`; `frontend/src/app/controller-telemetry/page.tsx`	LaTeX tables and the UI page, both read directly from the JSONs	`paper/tables.tex`, `paper/live_tab` `paper/main.pdf`; UI at route `/contr` `telemetry`

"What exactly is the input to the model? What comes out?"

- **Input:** 88 numbers per 5-second window of control-channel traffic. They describe only what the controller receives (PacketIn headers, FlowMod/stats counts). No host-side flow features, no payloads, no CSV features.
- **Output:** one decision per window, attack or not, plus recall, FPR and balanced accuracy with a 95% CI over 10 seeds.
- **What is NOT done:** the model is never given a flow's 5-tuple record from the CSV; the CSVs are used only to label windows.

"Where is everything stored?"

What	Where	In git?
Raw captures and archives	`data/raw/insdn_pcaps/`	no (large; hashes are recorded)
Window features	`research/insdn_controller/feats_v4/`	no (regenerable)
Result numbers (source of every figure)	`research/insdn_controller/results/*.json`	yes
Live-study raw pcaps	`research/insdn_controller/live/out*/`, `live/campaign2/`	partly (see `.gitignore`)
Capture hashes	`research/insdn_controller/capture_hashes.sha256`	yes
Paper source and PDF	`research/insdn_controller/paper/`	yes
Training device	NVIDIA RTX 4060 (CUDA): MLP, logistic regression and XGBoost all use `device=cuda`	n/a

Limitations that were fixed (and how), and those that were not

Limitation	Status
No tree-model baseline	Fixed: XGBoost on GPU, BAcc 0.952 ± 0.013 vs linear 0.961 ± 0.011 vs MLP 0.958 ± 0.012. Trees give no gain.

Limitation	Status
"Folds by capture, not scenario"	Already covered by experiment B (leave-one-attack-family-out) in `results/v4_strict_W5_xgb.json`; cite that instead of building a new split.
Frontend did not show the study	Fixed: `/controller-telemetry` page reads `/api/controller-telemetry`, which reads the result JSONs only.
Host-side feature comparison	Done (`results/hostside_xgb.json`), with the caveat that it is a different unit and protocol.
Source-side labelling for a future interleaved testbed	Removed from the repo during the paper cleanup (never run on data); it exists in git history under the tag `pre-paper-cleanup`.
Multi-switch topology, ≥10 runs per mode, cause of the 41 failed requests	Multi-switch (3-switch chain) and 10 runs per mode: **Done** (`--topo linear3`, 30 + 15 new runs). The 41-failure run did not reproduce (0 failures in 59 later runs); cause not established.
Second dataset or controller	**Partly done.** Second controller application: stock Ryu (done). Second dataset: UKMDDoSDN's two PCAPs downloaded and checked: 0 of 309,791 packets are OpenFlow, so unusable; LAN-SDN-NIDS data could not be reached (publisher blocks automated access) and is cited as related work only. OpenDaylight and ONOS under other apps are untested.
Published control-plane detector re-implemented	**Not done.** The comparison is to threshold heuristics and our own models.
Detector beating the trivial baseline in one session	**Not done.** Needs interleaved benign + attack traffic from the user's lab.
InSDN session-identity confound	Cannot be fixed (property of the benchmark); stated in the paper.

Extra hard questions about the data

Question	Answer
"Is the 'trivial rule' 0.5 or 0.995?"	Two different rules. "Any PacketIn" is always true on non-empty windows, so 0.5. "Any IP PacketIn" scores 0.995 (1 s) and 1.000 (5 s) within attack sessions under strict labels (`results/review_checks.json`), because the strict mask removes ambiguous windows and the leftover attack-free windows are ARP/LLDP only.
"Where do the labels come from, and are they trustworthy?"	From the InSDN CSV flow table, matched to the PACKET_IN's embedded frame. Coverage is partial, which is why windows with unmatched attack-like IPs are dropped under strict labels and counted as attack under the IP rule; both are reported.
"Why 5 s windows?"	1 s and 5 s are both run (`v4_strict_W1/W5.json`). 5 s has more PacketIns per window but fewer windows; conclusions do not change.
"Did you train on a CPU?"	No. CUDA only (project rule); MLP and logistic regression are PyTorch on the RTX 4060, XGBoost uses `device="cuda"`.
"Is any number made up?"	No. Every figure in the paper is read from `results/*.json`; `paper/verify_numbers.py` checks the quoted ones, and `reproduce.sh` regenerates the JSONs from the raw data.
"Can I rerun it?"	`research/insdn_controller/reproduce.sh` (GPU, raw InSDN captures per `data/MANIFEST.md`). Re-recording the live captures needs root (`README.md`).